

Request for Comments (RFC): DSpro — Agentic Tri-Modal Spatio-Temporal Video RAG System

Document ID: RFC-DSpro-FINAL

System: DSpro Agentic Spatio-Temporal RAG

Authors: Ramzi AL-ashmali

Date: June 17, 2026

1. Executive Summary

DSpro is an agentic, multi-modal video intelligence system that autonomously ingests raw video files (.mp4/.mkv) and indexes them for deep semantic retrieval.

Unlike traditional text-based RAG systems, DSpro achieves Joint Understanding across three dimensions—mathematically modeled as $V = f(\textit{Visual} + \textit{Audio} + \textit{Time})$. By treating video as a multidimensional database, the system allows users to submit complex, cross-lingual natural language queries (e.g., Arabic/English) and retrieves the exact start and end timestamps of specific events. This is achieved via a Late-Fusion architecture, hybrid vector search (Dense/Sparse), and an Agentic LLM Routing Layer, all engineered to operate strictly within edge-hardware constraints.

2. Hardware Constraints & Deployment Topology

The primary architectural constraint is the target deployment environment: a local machine with an NVIDIA RTX 2050 GPU and a rigid 4GB VRAM ceiling. This mandated a highly optimized, fragmented pipeline over end-to-end multi-modal models.

Deployment Architecture: Dockerized Microservices The system is fully containerized via Docker Compose to guarantee environment reproducibility (WSL 2 and native Linux).

- **Base Image Selection:** The standard python:3.11-slim was rejected. To support the C++ build tools and OS-level dependencies required by PyTorch and WhisperX, the system utilizes the official nvidia/cuda runtime base image.
- **Dedicated Vector Database:** Qdrant runs as a dedicated, standalone container (qdrant/qdrant) with persistent volumes. This isolates the database memory from the Python application memory, deprecating earlier file-based implementations.
- **GPU Pass-through:** Configured within docker-compose.yml via the deploy.resources.reservations.devices directive, granting the containerized FastAPI backend direct, low-latency CUDA access.
-

3. System Architecture & Data Flow

DSpro utilizes a Late-Fusion Architecture. Processing audio, visual frames, and text simultaneously (Early-Fusion) exceeds the 4GB VRAM limit. Instead, modalities are processed independently by specialized models and fused at the final retrieval stage using Reciprocal Rank Fusion (RRF).

3.1 The Agentic LLM Routing Layer (Gemini API) Before vector computation, queries pass through an agentic Gemini routing layer responsible for:

- **Cross-Lingual Normalization:** Translating complex Arabic queries into optimized English search vectors.
- **Fluff Stripping:** Removing conversational filler to isolate the core semantic intent.
- **Payload Generation:** Structuring an optimized Qdrant query with perfectly aligned dense semantic and sparse BM25 parameters.

3.2 The Search API (FastAPI) Strictness The /search endpoint natively calculates three strict metrics:

- **start_timestamp** (float)
- **end_timestamp** (float)

- **hybrid_rrf_score (float):** A normalized confidence match combining audio, visual, and text strengths. *(Note: UI metadata like video_id and thumbnail paths are appended to the JSON response without polluting the core mathematical backend logic).*

4. The Ingestion Engine (VRAM Paging Protocol)

To prevent Out-of-Memory (OOM) CUDA crashes, the pipeline uses a 4-Phase Sequential VRAM Paging Protocol. Models are dynamically loaded into VRAM, executed, and explicitly purged (`torch.cuda.empty_cache()`) before the next phase begins.

- **Phase 1: Audio Transcription & Alignment (WhisperX)** Extracts audio, transcribes speech, and aligns words to precise temporal timestamps.
- **Phase 2: Scene Boundary Detection (PySceneDetect)** Partitions the video based on visual cuts. Implements a 10-second micro-chunking fallback for monolithic scenes (e.g., long presentation slides).
- **Phase 3: Visual Text Extraction (EasyOCR)** Scans representative scene frames for on-screen text (e.g., code syntax, UI labels).
- **Phase 4: Vectorization**
 - *Dense Modality:* OpenCLIP (ViT-B-32) embeds visual frames into 512-d dense vectors.
 - *Sparse Modality:* BM25 (FastEmbed) vectorizes combined audio and OCR data. Operating entirely on the CPU, it provides exact keyword-matching without consuming GPU memory.

5. Vector Database & Retrieval Engine

Qdrant manages a multi-layered Hybrid Schema via Named Vectors, enabling simultaneous queries across disparate data types.

5.1 Schema Design

- **clip_visual:** Dense 512-d, Cosine Distance (semantic "look").
- **text_dense:** Dense 512-d, Cosine Distance (semantic "meaning" of speech/text).
- **text_sparse:** Sparse BM25 (exact lexical frequency of technical jargon).

5.2 Tri-Modal Reciprocal Rank Fusion (RRF)

The system executes three parallel vector searches and merges their ranks using RRF. This rewards segments that score highly across multiple modalities simultaneously.

- **Implementation:** 3 separate prefetch lists ($K = 60$) . Maximum possible score for a #1 rank across all three is $3 \times \left(\frac{1}{61}\right) \approx 0.0492$.
- **Hallucination Guard:** A hard threshold of 0.04 is enforced at the API level. Results scoring below this floor are rejected as Out-of-Distribution (OOD), preventing the Nearest Neighbor Fallacy.
-

6. Performance Metrics & Query Latency Budget

Tested on the uvicorn development server (PyTorch 2.11.0+cu128). Post-ingestion query performance is highly optimized..

6.1 Query Phase Latency Breakdown

Query Phase	Average Latency	Description
1. LLM Query Optimization	~400ms – 800ms	Network round-trip to Gemini API for translation/formatting.
2. Query Vectorization	~20ms	Local text encoding via OpenCLIP (local GPU execution).
3. Qdrant Hybrid RRF Search	~10ms	Parallel lookup and instant RRF mathematical fusion.
Total Turnaround Time	< 1 Second	From UI click to fully populated result card and video seek.

6.2 System Health Metrics

Extensive testing confirms the system's operational stability across all fronts, including frontend capabilities (thumbnails, progress bars, and persistent result cards are fully validated).

Metric	Target	Actual Performance	Status
VRAM Ceiling	< 4.0 GB	Zero OOM crashes across rigorous testing.	☑ Passed
API Stability	100% Uptime	All 8 endpoints return 200 OK consistently.	☑ Passed
Cross-Lingual OCR	Arabic/English	System accurately finds English code via Arabic prompts.	☑ Passed
Hallucination Rate	0% False Positives	Guard threshold (0.04) consistently filters OOD queries.	☑ Passed

7. Engineering Challenges & Resolutions

- 7.1 Navigating C++ Dependency Hell: Initial Docker builds failed due to missing C++ headers for audio libraries (TorchCodec, CTranslate2). Fix: Migrated to the NVIDIA CUDA base image and explicitly injected apt-get install -y ffmpeg into the Dockerfile, ensuring full-shared OS-level binaries were available for PyTorch.
- 7.2 LLM Routing Hallucinations: Complex Arabic queries occasionally resulted in poor target payloads, leading to irrelevant video retrieval. Fix:

Implemented a rigid `SYSTEM_PROMPT` as a Global Constant, sandboxing the LLM strictly to translation and vector-targeting instructions. Paired with the 0.04 RRF threshold, false positives were eliminated.

-

8. Conclusion & Future Work

Final Verdict: PRODUCTION-READY. DSpro successfully achieves its mandate: sub-1-second total turnaround for complex, cross-lingual, spatio-temporal video retrieval under strict 4GB VRAM hardware constraints.

Deliverables Completed:

- ☒ Autonomous, multi-modal Ingestion Engine (Audio, Visual, Text).
- ☒ High-speed Search API returning precise timestamps and integrated UI metadata.
- ☒ Fully portable Docker Compose GPU architecture.

Future Optimization Pipeline:

- **Vector Quantization:** Implement INT8 or fp16 quantization for OpenCLIP vectors to reduce VRAM and disk space requirements by up to 50% without significant cosine accuracy loss.
- **Full-Text Search Indexing:** Introduce a dedicated Qdrant payload index on transcript metadata to serve as an exact traditional textual substring search fallback.